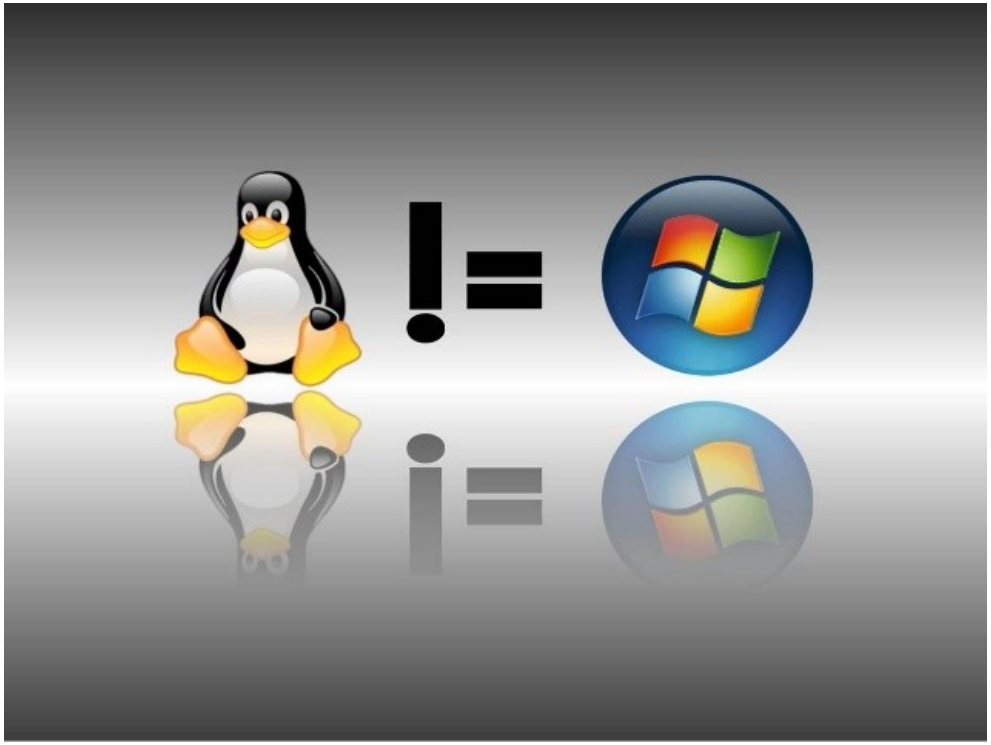
UNIX Introduction



Peter Wad Sackett

Why learn UNIX?

Automation

Stability

Development

Large tool set

Free(dom)

> Mastery of UNIX, like mastery of language, offers real freedom. The price of freedom is always dear, but there's no substitute. Personally, I'd rather pay for my freedom than live in a bitmapped, pop-up-happy dungeon like Windows. I'm hoping that as IT folks become more seasoned and less impressed by superficial convenience at the expense of real freedom, they will yearn for the kind of freedom and responsibility UNIX allows. When they do, UNIX will be there to fill the need. Thomas Scoville

Unix is command driven

We interact with Unix through a command line interface (CLI). Here we write what we want the computer to do. This sounds cumbersome, and it will start out that way, but as you learn, it gets more natural - and more importantly - more powerful.

WHAT we do:

There are many commands in Unix, which all do different things. They share a common way of getting input and delivering output. Because of that, they can be made to work together, similar to the way single words make up a language.

We will walk trough some of the more commonly used commands, and how they can be connected together to form a greater meaning or even programmatic structures.

WHERE we do it:

We will be working in the file system of the computer. This is a mass of files and folders, and we need to learn how to navigate this from the CLI and build up a mental landscape of the structure.
This is just a different view of the same structure seen from the desktop.

The Shell – command line

- Command line interface
- Everything has to be typed
- File system navigation
- Many simple tools available
- Tab completion
- Repeating commands
- Copy/Paste

```
robb@ubuntu: ~/test2
robb@ubuntu:~/test$ ls -l
total 12
-rw-r--r-- 1 robb robb 69 2011-10-04 23:04 file1
-rw-r--r-- 1 robb robb 88 2011-10-04 22:57 file2
-rw-r--r-- 1 robb robb 74 2011-10-04 23:32 file3
robb@ubuntu:~/test$ mv file3 ../test2
robb@ubuntu:~/test$ ls -l
total 8
-rw-r--r-- 1 robb robb 69 2011-10-04 23:04 file1
-rw-r--r-- 1 robb robb 88 2011-10-04 22:57 file2
robb@ubuntu:~/test$ cd ..
robb@ubuntu:~$ cd test2
robb@ubuntu:~/test2$ ls -l
total 8
-rw-r--r-- 1 robb robb 88 2011-10-04 23:45 file2
-rw-r--r-- 1 robb robb 74 2011-10-04 23:32 file3
robb@ubuntu:~/test2$
```

Understanding the notation

In this text and in the help you will find in several places, a certain notation is being used. A brief description is supplied.

Less than, greater than <>
The <word> is a mandatory element. It means that you must supply a word that is meaningful in the context of the command. It could be a file name or folder name, in which case the word enclosed in <> would be *file* or *folder*.

Square brackets []
When something is enclosed in [], the element is optional. You do not have to provide it. This is most often used for the options of the command, and is usually very literal.

Vertical bar |
This is used to show a choice between possibilities. Most often used inside [] or <>.

Three dots ...
This means the element can be repeated.

An example of a command description using the notation:
ls [-l] [file|folder]

File system navigation

Listing files

ls

ls [–l]

The path

Changing directories

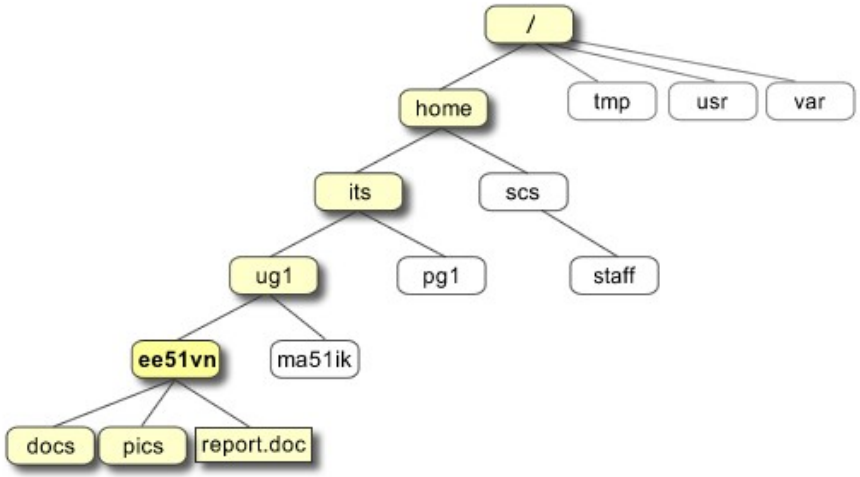
cd <folder>

Creating directories

mkdir <folder>

Deleting directories

rmdir <folder>



Where are you?

pwd

Why are you here?

Sorry, no answer to that one

File handling

Coping files

cp <file>… <destination>

Moving/renaming files

mv <file>… <destination>

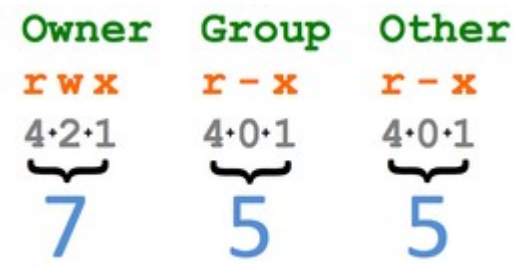
Deleting files

rm [-rf] <file>…

```
robb@ubuntu: ~/test2
robb@ubuntu:~/test$ ls -l
total 12
-rw-r--r-- 1 robb robb 69 2011-10-04 23:04 file1
-rw-r--r-- 1 robb robb 88 2011-10-04 22:57 file2
-rw-r--r-- 1 robb robb 74 2011-10-04 23:32 file3
robb@ubuntu:~/test$ mv file3 ../test2
robb@ubuntu:~/test$ ls -l
total 8
-rw-r--r-- 1 robb robb 69 2011-10-04 23:04 file1
-rw-r--r-- 1 robb robb 88 2011-10-04 22:57 file2
robb@ubuntu:~/test$ cd ..
robb@ubuntu:~$ cd test2
robb@ubuntu:~/test2$ ls -l
total 8
-rw-r--r-- 1 robb robb 88 2011-10-04 23:45 file2
-rw-r--r-- 1 robb robb 74 2011-10-04 23:32 file3
robb@ubuntu:~/test2$
```

Changing file permissions.

chmod <options> <file|folder>…



File inspection

Seeing the top of the file

head [-<number>] <file>

Seeing the end of the file

tail [-<number] <file>

Seeing all of the file

cat <file>...

Inspecting the file

less <file>

You can browse up and down in the file with **PgUp**/**b** and **PgDn**/**space**, search with **/**, but most importantly exit the application with **q**. **G** goes to the top and **g** to the bottom.

Some file manipulating

Counting the lines/words/bytes in the file

wc <file>…

Merging files

paste <file>…

Extracting columns from a file

cut <options> <file>

Usually used as ’cut –f2,4 myfile.txt’ on tab files

Sorting files

sort [options] <file>

Extracting lines from a file

grep <pattern> <file>

This is an incredibly useful command, very versatile.

IO redirection and pipes

Every program has **STDIN**, **STDOUT** and **STDERR**.
They are file streams – streams/lines of data and they are used for programs to communicate with each other or the file system.

Defaults; STDIN = keyboard, STDOUT, STDERR = screen

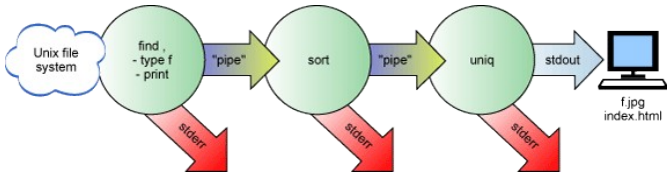
Save output of a command **in a file**; **>**

grep HUMAN orphans.sp > humanproteins.txt

Append to the file with **>>**

Feed a file to a command; **<**

wc < humanproteins.txt



Pipe (stream) the output of one command to the next command; **|**

grep HUMAN orphans.sp | wc

The pipe sign does not separate the commands, it connects them.

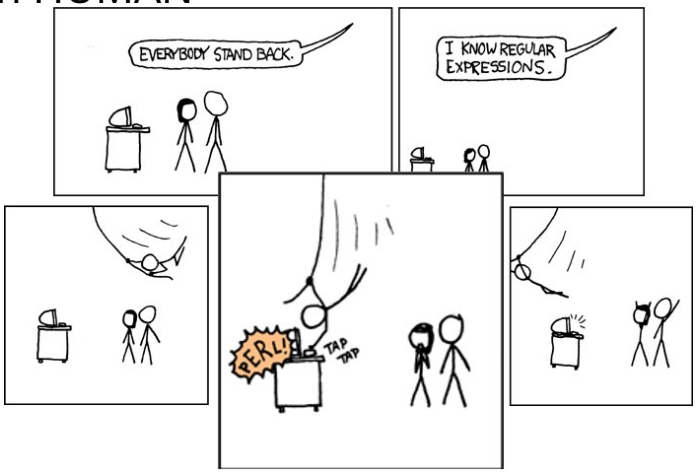
A closer look at grep and patterns

Many versions of grep – different capabilities

grep HUMAN <file>	Lines containing HUMAN, like POSTHUMAN
grep –v HUMAN	Lines without HUMAN
grep –c HUMAN	Count lines with HUMAN
grep "^HUMAN"	Lines starting with HUMAN
grep "HUMAN$"	Lines ending with HUMAN
grep –e/E	

Regular expressions

.	any single char
+	one or more of preceeding char
*	zero or more of preceeding char



grep –e "H.+MAN" Matches HEMAN, HUMAN, HITMAN, etc

Miscellaneous

Downloading files from the internet

wget <URL>

curl <URL> > <filename>

An URL is the link you see in the browser.

Printing text to the screen – or a file

echo <text>

Getting time and date information

date

A calculator

bc

Use ctrl D or quit to exit. Use it for calculations when you need a pocket calculator. It is a simple language that supports arbitrary precision numbers with interactive execution of statements.

File editing

Grahpical editors (using X)

Popular choices are Sublime Text and VScode.

There are a number of X-based editors, which can do the job:
nedit, gedit, jEdit, Kwrite, kate, TextWrangler (mac) and more.
Do a google search to see what is available.

Advantage: Easy to use.

Disadvantage: Requires good bandwidth on the network, when using them on remote servers. Your fave editor may not be available.

Text based editors

There are no popular choices as people generally do not like them.

nano, pico, vim, vi, emacs, Elvis, jove, ed.

Advantage: Some of these are alwas present on the system. They work very well on poor networks when accessing remote servers.

Disadvantage: Hard to learn.

More information and help

The UNIX manual

man <command>

The manual is shown with less. Also try google: man <command>

whatis <command>

Single line description of the command

Plenty of UNIX guides on the net. Here are some of the best.

http://www.ee.surrey.ac.uk/Teaching/Unix/

http://unixhelp.ed.ac.uk/

Google is your friend - sometimes.

Break

The rest of the powerpoint is for another day.

What will we learn?

- Accessing remote computers
- Controling your programs
- More commands dealing with files
- More about the shell – environment
- How to write small programs in bash (a shell).

There are a number of different shells. The most popular is bash.

All shell specific things are for the bash shell only.

Remote computers

Logging in to remote computers.

ssh -X <username>@<hostname>

This will start a shell on the remote computer if you have an account on the machine.

Transferring files to and from other computers.

ftp <hostname>
sftp <username>@<hostname>

Ftp is insecure and seldom used. Sftp is a secure alternative.
Give your username and password to the remote computer. You use the keywords "put" and "get" for file transfer.

scp <localfile> <username>@<hostname>:[remotepath]
scp <username>@<hostname>:<remotefile> <localpath>

Straightforward secure copying between computers. You need an account on both.

Is the remote machine alive?

ping <hostname>

Controlling programs

Checking what is running on the computer

top shows the most busy programs on the machine, q to quit

ps shows your programs

ps aux show all programs, more info

Terminating unruly programs

kill [-9] <processID>

Controlling a program running in your shell

ctrl C stops a program

ctrl Z suspends a program

bg sends a suspended program to the background for execution

fg a background program is send to the foreground

Executing a program in the background from the start

<someprogram> & the ambersand at the end is the key

Commands dealing with files

Creating a new and empty file

touch <filename>

Compressing and uncompressing files

zip <filename> check the manual before use

unzip <filename>

gzip <filename>

gunzip <filename>

Seeing the difference between two files

diff <file1> <file2>

Finding files

which <command> shows the path to the command

find <path> -name ”somename”

The find command is very complex and versatile. It could f.ex. be used to find files older than 10 days and delete them. Consult the manual before use.

Changing the content of files

sed

Substitute text or text pattern line by line in a file.

sed -i s/oldtext/newtext/ <filename>
sed 's/pattern/newtext/' <filename> > <newfilename>

sed uses a limited kind of regular expressions as patterns, and is both powerful and difficult to use. Definitely consult the manual.

awk or **gawk**

This command hides an entire language behind it, albeit it is quite primitive. It was designed to be able to do what you could not do in the shell, but without being a complete general purpose language like Python or Perl. It has been popular, but it's use have declined.

It is not to be underestimated to solving small file or line formatting problems, however people often turn to the tool they already know when solving problems.

Either consult the manual or just use Python.

Commanding the shell

What commands have you been doing

history [number_of_previous_commands_to_see]

Terminating the shell

logout

exit

ctrl D

Making your own command shortcut

alias <shortname> <longname>

The command path

echo $PATH

This is a list of folders where Unix will search for the commands you give. A new concept is demonstrated; an environment variable.
You can change this path, and thus change the commands available.

Executing your homemade programs/scripts not in the path

./<myscript> this executes the program in current dir

The environment - bash

All commands are executed in an environment. This is basically a list of specifically named variables, that has certain content, which can be used to control how execution of commands happens.

env shows all the environment variables

We have already met PATH which controls where to search for commands. Some standard variables are USER, HOME, PWD & SHELL

We can make our own variables in the shell – names are case specific

```
X=10
```

If the variables should be printed or otherwise used in the shell, they should be preceded with $ or even enclosed in ${}

```
echo ”X is” $X
echo ”X is” ${X}
```

The environment (aka the variables) are inherited to all programs executed by the shell. If a variable should become inheritable it must be exported first.

```
export X=10
```

Making shell scripts

A shell script is a program executed by the shell itself. This implies that anything that can be done in the shells command line, can be done in the script, and vice versa.

There are many different shells

•sh, the first original shell, Bourne Shell

•bash, the ”new” popular Bourne-Again Shell

•csh, the C Shell

•tcsh, the enhanced C Shell

•ksh, the Korn Shell

•zsh, the Z Shell

The different shells have different capabilities, however, in current time the most used is bash, and thus is the one used in the course.

The way to make a shell script is simple; edit a file, put in lines which the shell understands, i.e. Unix commands, maybe embellish with some shell-specific language constructs.

Shell script example in bash

```
#!/bin/bash
echo "Hello World"

# Create output file containing a date stamp
date > output.txt

# Showing the output file
cat output.txt
```

The first line is the shebang line. It is the two first chars **#!** that makes it into a shebang line. It MUST be the first line, if it should mean anything special. The following chars are ”just” the path to the bash shell (or other language) that should execute the rest of the file.

As can be seen, the unix commands **echo**, **date** and **cat** are being used in this script, together with one IO-redirection (**>**).

Furthermore the hash **#**, denotes as usual a comment line.

If this script was saved in the file *myscript.bsh*, then for the script to executable, the permissions of the file must allow for that.

```
chmod a+x myscript.bsh

./myscript.bsh
```

Bash shell language constructs

The bash shell has a surprising amount of language elements, that we know from higher level languages.

- Simple variables
- Lists
- Dicts (hashes, associative arrays)
- Value manipulations and expressions
- Conditional statements
- Loops
- Functions

As can be seen from this list, the bash contains the necessary elements for fairly advanced programming. You are not required to learn all this, but knowing some can be very useful.

Simple variables

There must be NO spaces on the line not enclosed in quotes. Bash treats everything as a string, also numbers.

```
num=10
str=Hello
hullo="Hello World"
```

If you want you use the value of a variable, prepend with **$**

```
echo $num $str
```

Calculation with "numbers" - use arithmetic expansion.
This means enclose the calculation with **$(( ))**

```
twicenum=$(($num+$num))
```

To see the difference (very instructive), try

```
dupnum=$num+$num
echo $twicenum $dupnum
```

There are no floating point calculations in bash, only simple integer calculations. If you want to use more advanced math, use external tools like **bc**. Calculations should be in assignments.

Lists and loops

Assigning a list, simple values enclosed in parentesis

```
numbers=(2 3 5 7 11 13)
```

Iterating through the list - syntax is awful

```
for number in ${numbers[@]}; do
    echo $number
done
```

```
# Summing the numbers
sum=0
for number in ${numbers[@]}; do
   sum=$(($sum+$number))
done
echo $sum
```

```
# Running a program on a list of filenames
$program="wc -l"
files=(file1.txt file2.txt file3.txt file3.txt)
for file in ${files[@]}; do
    echo -n "Lines in $file: "
    $program $file
done
```

Getting results from commands

Often you want to know the result of an external command and use that in your script. Use the backticks ` to enclose the command.

```
# Looking for files matching a pattern in current dir
genbankfiles=`ls ./data*.gb`
echo $genbankfiles
for file in $genbankfiles; do
   echo "Genbank file is: " $file
done
```

The $genbankfiles variable is a simple string - not a list, but the spaces in it works as delimiters between elements. This is a good reason why it is BAD to have spaces in file names. The file name gets to be split up.

Sometimes you just want to know if a command failed or not. EVERY command in Unix has an exit code, where 0 means success and non-zero means failure. The non-zero failure is command-specific, and if you want to know the exact failure, consult the command's manual.
You must get the exit code right after the execution of the command.

```
<someprogramorcommand>
exitcode=$?
```

Conditional statements

The classic **if** statement is used for testing for a condition. The **elif** and **else** are optional, but **fi** is not.

```
if test1; then
   <somecommands>
elif test2; then
   <othercommands>
else
   <somecommands>
fi
```

The condition test is unfortunately both complex and versatile, meaning it can do a lot (file tests, string pattern matches, apart from normal simple equality/greater-than test), so it is easy to get lost in.

The basic test is enclosed in double brackets; **[[** expression **]]**
They can be put together with **&&** (and) and **||** (or).

```
if [[ $exitcode == 0 ]]; then
   echo "Command succeded"
else
   echo "Failed"
fi
```

Command line arguments

The bash shell can be given command line arguments. They will be found in the variables **$1**, **$2** ... up to **$9** according to their position. The arguments can also be found in the list **$@**.

```
pattern=$1
if [[ $pattern == "" ]]; then
   echo "Missing file pattern argument"
   exit 1
fi
program="wc -l"
# Variables can be used in backticks
# 2> redirects STDERR, here to the trash can
files=`ls $pattern 2> /dev/null`
exitcode=$?
if [[ $exitcode != 0 ]]; then
   echo "No files match the pattern"
   exit 0
fi
for file in ${files[@]}; do
   echo -n "Lines in $file: "
   $program $file
done
```

Your future bash programming

Bash has much more to offer than show here. There are many resources on the net and the official manual may not be the best place to start.

https://www.gnu.org/software/bash/manual

There are more looping constructs: **while**, **until**, and even **break** and **continue** works.

Conditional constructs also include **case** and **select**.

Regular expression matches has not been touch upon, neither the complexity of string slicing and part matching, nor functions and process control.

In short, this was a very brief introduction to shell programming, and bash is a good tool to turn to, when you just want something simple.

Just explore